

UNIVERSIDAD AUTÓNOMA METROPOLITANA

Unidad Cuajimalpa

Laboratorio Temático III

Documentación del Modelo Relacional

Plataforma de Distribución de Videojuegos

Autores

Ramos Reyes Aaron Rodrigo

Cruz Lopez Ana Lilia Guadalupe

Alan Reyes Rosales

2 de octubre de 2026

1. Link Repositorio

- **Repositorio GitHub:** https://github.com/Mort-Mort-source/proyecto_tematicoIII.git

2. Introducción

El presente proyecto consiste en el diseño e implementación de una base de datos relacional para una plataforma de distribución digital de videojuegos y software, inspirada en el modelo de negocio y funcionalidades de Steam. El objetivo principal es proveer una estructura robusta, escalable y libre de anomalías que permita gestionar el ciclo de vida completo del usuario en la plataforma: desde su registro e interacción social, hasta la exploración del catálogo, el procesamiento de compras y la gestión de su biblioteca personal de títulos. Esta arquitectura de datos servirá como la capa de persistencia para una futura API REST.

3. Flujo de Uso del Sistema

El diseño de la base de datos soporta de forma nativa el siguiente flujo de interacción del usuario:

1. **Autenticación e Identidad:** El visitante se registra creando un `Usuario`. A partir de ese momento, puede establecer conexiones sociales agregando a otros usuarios a su lista de `Amigos`.
2. **Exploración del Catálogo:** El usuario navega por los `Productos` disponibles, los cuales están clasificados por múltiples `Categorías` y asociados a un `Desarrollador`.
3. **Intención de Compra:** Si un usuario tiene interés en un producto, puede añadirlo a su lista de `Deseados` para un seguimiento a futuro, o agregarlo al `Carrito` especificando las cantidades.
4. **Proceso de Checkout:** El sistema consolida los elementos del carrito en una `Orden` de compra, desglosando los elementos en el `Detalle_de_Orden`.
5. **Transacción:** Se genera una `Transacción` vinculada a un `Método_de_Pago` (ej. Tarjeta de Crédito, PayPal). Si la transacción es aprobada, el flujo avanza.
6. **Aprovisionamiento:** Los productos adquiridos se transfieren a la `Biblioteca` del usuario de forma permanente.
7. **Interacción Post-Compra:** Una vez en su biblioteca, el usuario acumula horas de juego y puede redactar `Reseñas` públicas, recomendando o no el producto.

4. Explicación de las Tablas y sus Relaciones

Para que el sistema funcione sin errores, la información no se guarda toda junta, sino que se divide en distintas tablas que se conectan entre sí. A continuación, explicamos cada relación de forma sencilla:

4.1 Catálogos y Entidades Base

- **Usuario:** Guarda la información de las personas registradas (correo, contraseña, teléfono).
- **Producto:** Guarda los detalles básicos de los videojuegos (nombre, precio, imagen).
- **Desarrollador y Categoría:** Son listas independientes que nos ayudan a clasificar los juegos sin escribir los nombres a mano cada vez, evitando errores de dedo.
- **Método de Pago:** Lista con las opciones habilitadas para pagar (tarjeta, PayPal, transferencias, etc.).

4.2 ¿Cómo se relacionan los Usuarios con los Productos?

Como un usuario interactúa con los juegos de muchas maneras, usamos tablas intermedias para conectar la tabla **Usuario** con la tabla **Producto**:

- **Biblioteca (Muchos a Muchos):** Un usuario compra muchos juegos, y un juego es adquirido por muchos usuarios. Esta tabla los une y guarda el precio exacto que el usuario pagó y la fecha en que lo consiguió.
- **Carrito (Muchos a Muchos):** Funciona como la canasta del supermercado. Conecta al usuario con los juegos que está a punto de comprar, guardando también cuántas copias (cantidad) quiere de cada uno.
- **Deseados (Muchos a Muchos):** Es la lista de deseos. Conecta al usuario con los juegos que le interesan para el futuro, guardando la fecha en que los agregó.
- **Reseñas (Muchos a Muchos):** Un usuario puede opinar sobre varios juegos, y un juego recibe opiniones de muchos usuarios. Aquí guardamos su comentario, si lo recomienda o no, y cuántas horas ha jugado.

4.3 Otras Relaciones Importantes

- **Amigos (Usuario con Usuario):** Un usuario puede tener muchos amigos en la plataforma. Usamos esta tabla para conectar a un Usuario con otro Usuario y guardar la fecha en la que se conectaron.
- **Producto - Desarrollador (Muchos a Uno):** Un estudio creador (como Epic Games o Valve) hace muchos juegos. Cada juego se conecta a su creador a través del ID del desarrollador.
- **Producto - Categoría (Muchos a Muchos):** Un juego puede ser de “Aventura” y “Acción” al mismo tiempo, y la categoría “Aventura” abarca muchos juegos. La tabla `Producto_Categoria` se encarga de hacer estos múltiples enlaces.

4.4 El Proceso de Compra (Órdenes y Transacciones)

Para llevar un control financiero exacto, las ventas se estructuraron de esta manera:

- **Usuario - Orden (Uno a Muchos):** Un usuario puede hacer muchas compras a lo largo del tiempo. Cada compra genera un “ticket” general llamado `Orden`.
- **Orden - Detalle de Orden (Uno a Muchos):** La `Orden` es el total a pagar, pero el `Detalle_de_Orden` son los “renglones” de ese ticket.
- **Producto - Detalle de Orden (Uno a Muchos):** Cada “renglón” del ticket se conecta directamente con un `Producto` para saber exactamente qué juego se vendió en esa compra y a qué precio unitario.
- **Orden - Transacción (Uno a Muchos):** Para pagar una `Orden`, se hace un intento de cobro llamado `Transacción`. Aquí anotamos si el cobro fue “Aprobado”, “Rechazado” o “Reembolsado”.
- **Transacción - Método de Pago (Muchos a Uno):** Cada intento de cobro se enlaza a un método de pago de nuestro catálogo para saber si el cliente usó tarjeta, saldo de cartera, etc.

5. ¿Por qué se organizó así? (Reglas de Normalización)

El diseño de esta base de datos sigue reglas profesionales para evitar que la información se duplique, se vuelva contradictoria o se pierda. En una presentación, esto significa:

- **Cada dato en su lugar (1NF):** Evitamos meter mucha información mezclada en un solo espacio. Por ejemplo, en lugar de escribir un párrafo revuelto con los requisitos de la computadora (memoria, procesador), usamos un formato estructurado (JSON) para que el sistema lo lea fácilmente.
- **Dependencia directa (2NF):** En las tablas que unen dos elementos (como un `Usuario` y un `Producto` en el `Carrito`), los datos adicionales como la “Cantidad” dependen exclusivamente de ese cruce en particular, no del usuario por separado ni del juego por separado.
- **Evitar la repetición de datos (3NF):** Si guardáramos el nombre del creador (*Desarrollador*) directamente dentro del juego y ese estudio cambiara de nombre, tendríamos que buscar y actualizar miles de juegos uno por uno. Al separar al desarrollador en su propia tabla, solo modificamos el nombre una vez y todos sus juegos se actualizan en cascada de forma automática.

6. Diagrama de la Base de Datos

A continuación se presenta el Modelo Entidad-Relación (EER) generado a partir de la implementación en MySQL, el cual ilustra de manera visual todas las tablas, atributos y las conexiones descritas a lo largo de este documento.

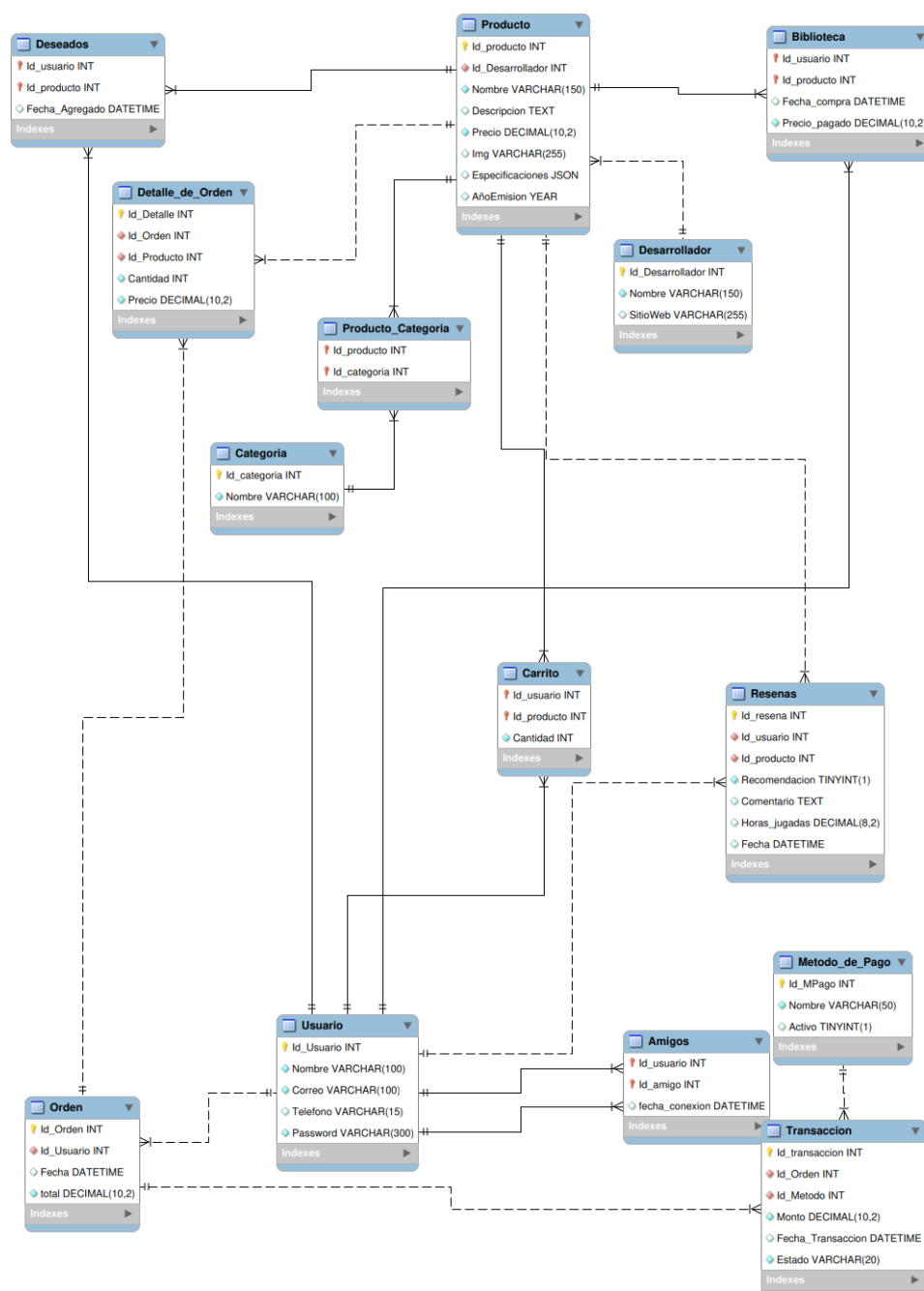


Figura 1. Modelo Entidad-Relación de la Plataforma